GET STARTED
Welcome to SP-API Documentation
What is the Selling Partner API?
Selling Partner API Onboarding Overview
Terminology
Frequently Asked Questions

CHANGELOG
SP-API Release Notes
SP-API Deprecation Schedule
SP-API Product Metadata Updates

DIRECTORIES

# Tutorial: Automate your SP-API Calls Using a Java SDK

Automate your SP-API calls with Login with Amazon (LWA) token exchange and authentication.

---

This tutorial describes how to generate a Java SDK with Login with Amazon (LWA) token exchange and authentication to build your application seamlessly. You will learn the prerequisites required to build the Java SDK and also view an example using the Selling Partner API (SP-API) for sellers and Swagger Code Generator.

First, take a look at how the authorization process works.

## Prerequisites

To complete this tutorial, you need the following prerequisites:

- A hybrid or SP-API app in draft or published state
- Integrated development environment (IDE) software (this walkthrough uses Eclipse IDE on Windows OS)

You must register your application before it can connect to the Selling Partner API, and it must be authorized by a selling partner. If you do not have a hybrid or SP-API app, follow the steps to register as a developer, register your application, and Authorizing Selling Partner API applications. Then, return to this tutorial.

Next, set up your workspace for the tutorial.

## Step 1. Set up your workspace

You must download the required software packages and the Swagger Code Generator. Then, clone the SP-API models repository.

1. On your local drive, create a directory for this project and name it `SwaggerToCL`.
2. Download the following tools and make them available in your `$PATH`.
    1. `Java 17` or newer or `OpenJDK 17` or newer
    2. `Apache Maven` 3.9 or greater
    3. `GNU Wget` 1.20 or greater
3. Run the following command to download the Swagger Code Generator:

Bash

```
wget
https://repo1.maven.org/maven2/io/swagger/codegen/v3/swagger-
```

SP-API Models
SP-API Seller Use Cases
SP-API Vendor Use Cases
Seller Central URLs
Vendor Central URLs

REGISTRATION

SP-API Registration Overview

Developer Registration Request Status
Third-Party Provider Registration

Service Provider Registration and Role Approval Process Overview
Website Guidelines for Public Developers and Service Providers
User Permissions for Service Providers
Register your Application
Update your Application Information
Rotate your Application's LWA Credentials

View your Application Information and Credentials
Learn How Sellers Authorize Service Providers
Selling Partner API Roles

Policies and Agreements
About Facial Data

AUTHORIZATION

Authorize Applications

Renew Authorizations
Revoke Authorizations
Authorization Limits
Special Authorization Types

INTEGRATION

```
codegen-cli/3.0.61/swagger-codegen-cli-3.0.61.jar
```

4. Copy `swagger-codegen-cli.jar` into your local directory `C:\\SwaggerToCL`.

5. In GitHub, go to https://github.com/amzn/selling-partner-api-models/tree/main/models and use the following command to clone the **`selling-partner-api-models`** repository to your local directory **`C:\\SwaggerToCL`**.

Bash

```
git clone https://github.com/amzn/selling-partner-api-models
```

6. Navigate to the **`selling-partner-api-models\models`** folder in your local copy of the repository and copy all JSON files from the models subfolders into **`C:\\SwaggerToCL`**.

   The `selling-partner-api-models` repository has two directories:

   1. models directory - contains all the currently available Selling Partner API models.

> **Note**
>
> Developers use Swagger Codegen to generate client libraries from these models. Swagger CodeGen is an open source project which enables automatic generation of API client libraries (SDK generation), server stubs, and documentation from an OpenAPI specification. For more information, refer to Swagger Codegen.

   1. clients directory - contains a Java library named `sellingpartner-api-aa-java` and a C# library named `sellingpartner-api-aa-csharp` with Mustache templates for use with `swagger-codegen` to generate client libraries with authentication and authorization functionality included. The `sellingpartner-api-documents-helper-java` folder contains helper classes for encrypted documents. The `sample-code` folder contains java code examples for the Restricted Data Token (RDT).

Now that you have completed the required setup, the next step is to generate the Java SDK with the authentication and authorization classes provided in the `sellingpartner-api-aa-java` folder.

## Step 2. Generate Java SDK for SP-APIs

In this step, you generate the SDK against the templates in the folder **`selling-partner-api-models\clients\sellingpartner-api-aa-java`** in your local copy of the repository. This folder contains an authorization and authentication library, along with customized templates for the Swagger Code Generator, and a config file to generate the correct version of the SDK.

To generate the Java SDK, you need to run the following command for each JSON file in your local directory:

Bash

```
java -jar C:\SwaggerToCL\swagger-codegen-cli.jar generate \
-i C:\SwaggerToCL\[name of model].json \
-l java -t [path to selling-partner-api-
models\clients\sellingpartner-api-aa-java folder]\resources\swagger-
codegen\templates\ \
-o C:\SwaggerToCL\[name of client library] \
```

Building Robust Amazon SP-API Applications
SP-API SDKs
Delete an Application From Your Developer Account
CALLING THE SP-API
Configuration Details
Call Structure
Development Tools
Performance Management
SOLUTION PROVIDER PORTAL
Manage Users in Solution Provider Portal
Manage Your Business And Contact information in Solution Provider Portal
Update Your Developer Profile in Solution Provider Portal
Unify Your Developer Accounts
Verify Your Identity in Solution Provider Portal

```
-c [path to selling-partner-api-models\clients\sellingpartner-api-aa-java folder]\resources\java\config.json
```

For example, run the following command to generate Java code for **`sellers.json`**.

Bash

```
java -jar C:\SwaggerToCL\swagger-codegen-cli.jar generate \
-i C:\SwaggerToCL\Sellers.json \
-l java -t [path to selling-partner-api-models\clients\sellingpartner-api-aa-java folder]\resources\swagger-codegen\templates\ \
-o C:\SwaggerToCL\Sellers_JavaCL \
-c [path to selling-partner-api-models\clients\sellingpartner-api-aa-java folder]\resources\java\config.json
```

Repeat this step to generate SDKs for each of the SP-APIs available in the models directory.

The generated SDK includes the classes for configuring your LWA credentials and uses them to exchange LWA tokens.

In the following steps, **x.x** refers to the latest version of the AA library. Refer to the readme to get the latest version.

## Step 3. Build the Authentication/Authorization Library (AA)

After you generate the SDK, build the AA library.

1. Navigate to the **`selling-partner-api-models\clients\sellingpartner-api-aa-java`** folder of your local copy of the repository and run `mvn package`. This generates a folder named `target`. In this folder is a `JAR` file named **`sellingpartnerapi-aa-java-x.x-jar-with-dependencies.jar`** (or similarly named) and all of the required dependencies.

2. Run the following command to install the `JAR` file in your local Maven repository.

   mvn

   ```
   mvn install:install-file -Dfile=[path to JAR file in "target" folder] -DgroupId=com.amazon.sellingpartnerapi -DartifactId=sellingpartnerapi-aa-java -Dversion=x.x -Dpackaging=jar
   ```

   You can find the actual `groupId`, `artifactId`, and `version` values near the top of the **`pom.xml`** file in the **`selling-partner-api-models\clients\sellingpartner-api-aa-java`** folder.

   Example:

   mvn

   ```
   mvn install:install-file -Dfile=C:\SwaggerToCL\sellingpartner-api-aa-java\target\sellingpartnerapi-aa-java-x.x.jar -DgroupId=com.amazon.sellingpartnerapi -DartifactId=sellingpartnerapi-aa-java -Dversion=x.x -Dpackaging=jar
   ```

   mvn

API Usage Metrics in Solution Provider Portal

TUTORIALS

Tutorial: Subscribe to the ORDER_CHANGE Notification

Tutorial: Test Selling Partner API Endpoints

Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a Java SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js

Tutorial: Automate your SP-API Calls Using a Python SDK

Tutorial: Automate your SP-API Calls using a prebuilt C# SDK

Tutorial: Automate your SP-API Calls using prebuilt Java SDK

Tutorial: Automate your SP-API Calls using a prebuilt JavaScript SDK

Tutorial: Automate your SP-API Calls using prebuilt PHP SDK

Tutorial: Automate your SP-API calls using a prebuilt Python SDK

Tutorial: Authorize Multiple Vendor Central Accounts with a Single SP-API Application

Tutorial: Retrieve Merchant Shipping Templates

Tutorial: Grant the SP-API Permission to an Amazon SQS Queue

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

TROUBLESHOOTING

Troubleshoot SP-API Errors

URL Encoding

Resolve Common HTTP and Authorization Error Codes

External Fulfillment Errors

Listings Items API Issues Troubleshooting

SELLING PARTNER APPSTORE

What is the Selling Partner Appstore?

List Your App on the Selling Partner Appstore

Edit Your Appstore Listing

Check Listing Status

Appstore Ratings and Reviews

Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview

```
mvn install:install-file -Dfile=C:\SwaggerToCL\sellingpartner-api-aa-java\target\sellingpartnerapi-aa-java-x.x-jar-with-dependencies.jar -DgroupId=com.amazon.sellingpartnerapi -DartifactId=sellingpartnerapi-aa-java -Dversion=x.x -Dpackaging=jar
```

## Step 4. Add the library as a dependency of the SDK

To add the library as a dependency, you need to edit the `pom.xml` file.

1. In your local directory, navigate to the generated Seller API SDK and open the `pom.xml` file.
2. Add the following code as a dependency in the file.

XML

```
<dependency>
<groupId>com.amazon.sellingpartnerapi</groupId>
<artifactId>sellingpartnerapi-aa-java</artifactId>
<version>x.x</version>
</dependency>
```

> **Note**
>
> **x.x** refers to the latest version of the AA library. Refer to the readme to get the latest version.

3. Save and close the file.

The next step is to write the actual application and connect to the SP-API using the generated Java SDK.

## Step 5. Connect to the Selling Partner API using the generated Java SDK

In this step, you connect to the Selling Partner API using the Java SDK generated in the preceding steps. These steps use Eclipse IDE to import the generated SDK and connect to the Selling Partner API, but you can use your preferred IDE.

1. In Eclipse IDE, on the **File** menu, choose **Open Projects from File System** and then import the generated Java SDK.
2. In `src/main/java` add a `class` to write your code.
3. Enter code in the added `class` to configure the following instance:
    1. LWA credentials (refer to Configure your LWA credentials)

> **Note**
>
> If the class path doesn't recognize the classes in Eclipse, you'll need to add the `JAR` files that were created in the `target` folder, which is located in the generated SDK folder.

4. Create an instance of the generated SDK API and call the operations. For an example, refer to the following sample code for connecting to the Selling Partner API for Sellers (provided that you generated the SDK for this API).

Java

Amazon Selling Partner API Guard Implementation Guide

VAT Calculation Service

Amazon Seller Data Access Best Practices

A+ CONTENT API

A+ Content API

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API

Amazon Warehousing and Distribution API

```
import io.swagger.client.*;
import io.swagger.client.auth.*;
import io.swagger.client.model.*;
import io.swagger.client.api.SellersApi;
import java.io.File;
import java.util.*;
import
com.amazon.SellingPartnerAPIAA.LWAAuthorizationCredentials;
import com.amazon.SellingPartnerAPIAA.LWAException;
import static
com.amazon.SellingPartnerAPIAA.ScopeConstants.SCOPE_NOTIFICATIONS_API;
// for grantless operations (Notifications API)
import static
com.amazon.SellingPartnerAPIAA.ScopeConstants.SCOPE_MIGRATION_API;
// for grantless operations (Authorization API)

public class SellersApiDemo {

    public static void main(String[] args) {

        //Configure your LWA credentials :
        LWAAuthorizationCredentials lwaAuthorizationCredentials =
LWAAuthorizationCredentials.builder()
                .clientId("amzn1.application-
**********************")

.clientSecret("************************************")

.refreshToken("Atzr|*****************************")
                .endpoint("https://api.amazon.com/auth/o2/token")
                .build();

        //For Grantless operations (Authorization API and
Notifications API), use following code :
                /* LWAAuthorizationCredentials
lwaAuthorizationCredentials =
LWAAuthorizationCredentials.builder()
                .clientId("amzn1.application-
**********************")

.clientSecret("************************************")
                .withScopes(SCOPE_NOTIFICATIONS_API,
SCOPE_MIGRATION_API)
                .endpoint("https://api.amazon.com/auth/o2/token")
                .build(); */

        //Create an instance of the Sellers API and call an
operation :
        SellersApi sellersApi = new SellersApi.Builder()

.lwaAuthorizationCredentials(lwaAuthorizationCredentials)
                .endpoint("https://sellingpartnerapi-
na.amazon.com") // use Sandbox URL here if you would like to test
your applications without affecting production data.
                .build();

        try {
            GetMarketplaceParticipationsResponse result =
sellersApi.getMarketplaceParticipations();
            System.out.println(result);
        } catch (ApiException e) {
            System.err.println("Exception when calling
SellersApi#getMarketplaceParticipations");
            e.printStackTrace();
        }
        catch (LWAException e) {
```

Now that you have run the code, run the `main` class. You now have a list of marketplaces that your authorized seller can sell in.

You can follow similar steps for other SP-APIs.

```
            System.err.println("LWA Exception when calling
        SellersApi#getMarketplaceParticipations");
            System.err.println(e.getErrorCode());
            System.err.println(e.getErrorMessage());
            e.printStackTrace();
        }
    }
}
```

Updated 18 days ago

← Tutorial: Automate your SP-API Calls Using a C# SDK

Tutorial: Automate your SP-API Calls Using a JavaScript SDK for Node.js →



Amazon Warehousing and Distribution API v2024-05-09 Reference

APP INTEGRATIONS API

App Integrations API

Policies and Agreements

Privacy notice | Conditions of use

APPLICATION MANAGEMENT API

Application Management API

CATALOG ITEMS API

Catalog Items API

Catalog Items API Rate Limits

Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API

Customer Feedback API Rate Limits

DATA KIOSK

Data Kiosk API

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API

EASY SHIP API

Easy Ship API

EXTERNAL FULFILLMENT

External Fulfillment Inventory API

External Fulfillment Returns API

External Fulfillment Shipping API

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API

FBA Inventory Dynamic Sandbox Guide

FEEDS API

Feeds API

Feed Type Values

Feeds JSON Schemas

Feeds API Best Practices

Feeds API FAQ

Feeds API Rate Limits

FINANCES

Finances API

Transfers API

FULFILLMENT INBOUND API

Fulfillment Inbound API

Fulfillment Inbound API FAQ

Migrating Fulfillment Inbound workflows

Fulfillment Inbound API Rate Limits

FULFILLMENT OUTBOUND API

Fulfillment Outbound Dynamic Sandbox Guide

Fulfillment Outbound API

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API

Invoices API FAQ

LISTINGS

Listings Items API

Listings Items API Rate Limits

Listings Restrictions API

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API >

MESSAGING API

Messaging API >

NOTIFICATIONS API

Notifications API >

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification ↗

ORDERS API

Orders API >

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API

PRODUCT PRICING API

Product Pricing API

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide

Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API

REPORTS API

Reports API

Report Type Values

Reports JSON Schemas

Reports API FAQ

SALES API

Sales API

SELLER WALLET API

Seller Wallet API

Seller Wallet API Rate Limits

SELLERS API

Sellers API

SERVICES API

Services API

SHIPMENT INVOICING API

Shipment Invoicing API

SHIPPING API

Shipping API v2 Resources

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API

SUPPLY SOURCES API

Supply Sources API >

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API >

UPLOADS API

Uploads API >

VEHICLES API

Vehicles API >

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API >

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API >

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API >

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API >

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API >